# 🐚 Mémo Bash — BTS SIO 2, option SISR

> Le pense-bête du scripteur : tout ce qu'il faut avoir sous la main pour écrire un script bash propre, sans googler "comment tester si un fichier existe" pour la 47e fois.

---

## 1. Les bases du script

```
#!/bin/bash # Shebang : OBLIGATOIRE en 1ère ligne
# commentaire # Le reste de la ligne est ignoré
chmod +x monscript.sh # Rendre le script exécutable
./monscript.sh # L'exécuter
bash monscript.sh # Ou comme ça (pas besoin de chmod)
```

💡 **Réflexe pro** : `bash -x monscript.sh` exécute le script en mode debug et affiche chaque commande. Ton meilleur ami quand ça ne marche pas.

---

## 2. Variables

```
nom="Alice" # PAS d'espace autour du =
echo "Bonjour $nom" # Affichage
echo "Bonjour ${nom}e" # Les {} délimitent le nom de variable
readonly VERSION="1.0" # Constante : non modifiable
unset nom # Supprime la variable
```

### Variables spéciales

| Variable | Signification |
|---|---|
| `$0` | Nom du script |
| `$1`, `$2`, … | Arguments passés au script |
| `$#` | Nombre d'arguments |
| `$@` | Tous les arguments (un par un) |
| `$?` | Code retour de la dernière commande (0 = OK ✅) |
| `$$` | PID du script |
| `$USER`, `$HOSTNAME`, `$PWD` | Environnement classique |

### Lecture au clavier

```
read -p "Ton prénom : " prenom
read -s -p "Mot de passe : " mdp # -s : saisie masquée
```

### Calculs

```
total=$(( 5 + 3 * 2 )) # Arithmétique entière
let "compteur = compteur + 1"
(( compteur++ )) # Incrément
echo $((RANDOM % 10)) # Nombre aléatoire 0-9
```

⚠️ Bash ne sait pas faire de décimales : pour ça, `bc` ou `awk`.

---

## 3. Tests : `if`

```
if [ condition ]; then
 echo "vrai"
elif [ autre_condition ]; then
 echo "moyen"
else
 echo "faux"
fi
```

### Tests sur les fichiers (le pain quotidien du SISR 🩹)

| Test | Vrai si… |
|---|---|
| `-e fichier` | existe |
| `-f fichier` | existe et est un fichier |
| `-d dossier` | existe et est un répertoire |
| `-r` / `-w` / `-x` | lisible / inscriptible / exécutable |
| `-s fichier` | existe et n'est pas vide |
| `f1 -nt f2` | f1 plus récent que f2 |

### Tests sur les chaînes et les nombres

| Test | Signification |
|---|---|
| `-z "$s"` / `-n "$s"` | chaîne vide / non vide |
| `"$a" = "$b"` | chaînes égales |
| `"$a" != "$b"` | différentes |
| `-eq`, `-ne` | égal, différent (nombres) |
| `-lt`, `-le`, `-gt`, `-ge` | <, ≤, >, ≥ (nombres) |

💡 **Règle d'or** : toujours mettre les variables **entre guillemets** dans les tests : `[ -f "$fichier" ]`. Sinon, une variable vide fait exploser le test.

**La version moderne : `[[ ]]`**

```
if [[ "$nom" == A* ]]; then # motifs (glob)
if [[ "$age" -gt 18 && "$age" -lt 65 ]]; then # && et || directement
```

---

## 4. Case : le "switch" de bash

```
read -p "Choix : " choix
case "$choix" in
 1) echo "Sauvegarde" ;;
 2) echo "Restauration" ;;
 q|Q) echo "Au revoir !" ;;
 *) echo "Choix invalide" ;; # * = par défaut
esac
```

Parfait pour les menus d'administration. 🎛️

---

## 5. Boucles

```
# For sur une liste
for fichier in /var/log/*.log; do
 echo "Traitement de $fichier"
done
# For compteur
for i in {1..10}; do
 echo "Tour n°$i"
done
for i in $(seq 0 5 100); do echo "$i"; done # 0,5,10...100
# While : tant que
while [ $compteur -lt 10 ]; do
 (( compteur++ ))
done
# While + read : ligne par ligne (indispensable avec les logs)
while read -r ligne; do
 echo "$ligne"
done < /etc/passwd
# Until : jusqu'à ce que
until ping -c1 192.168.1.1 &>/dev/null; do
 echo "Réseau indisponible, nouvelle tentative..."
 sleep 2
done
```

---

## 6. Fonctions

```
function sauvegarde() { # ou simplement : sauvegarde() {
 local dest="$1" # $1 = 1er argument DE LA FONCTION
 echo "Sauvegarde vers $dest"
 return 0 # code retour (0 = succès)
}
sauvegarde /backup # Appel SANS parenthèses
```

💡 `local` évite que la variable pollue le reste du script. Une fonction = une seule tâche.

---

## 7. Sorties, redirections et pipes

| Opérateur | Effet |
|---|---|
| `>` | Redirige stdout (écrase) |
| `>>` | Redirige stdout (ajoute à la fin) |
| `2>` | Redirige stderr (les erreurs) |
| `2>&1` | Redirige stderr vers stdout |
| `&>` | stdout + stderr ensemble |
| `<` | Redirige stdin |
| ` | ` |
| `/dev/null` | Le trou noir 🕳️ (jeter une sortie) |

```
./script.sh >> /var/log/monscript.log 2>&1 # Tout logger
commande > /dev/null 2>&1 # Silence total
ls -l | grep ".conf" | wc -l # Chaînage (pipe)
```

### Substitution de commande

```
date_aujourdhui=$(date +%Y%m%d) # $( ) = résultat d'une commande
nb_users=$(wc -l < /etc/passwd)
```

---

## 8. Commandes incontournables à combiner

| Commande | Rôle |
|---|---|
| `grep` | Chercher (options : `-i` sans casse, `-v` inverser, `-c` compter, `-r` récursif) |
| `awk` | Extraire des colonnes : `awk '{print $1}'` |
| `cut` | Autre découpage : `cut -d: -f1 /etc/passwd` |

| Commande | Rôle |
| --- | --- |
| `sed` | Remplacer : `sed 's/ancien/nouveau/g' fichier` |
| `sort` / `uniq` | Trier / dédoublonner (`sort fichier |
| `wc` | Compter (`-l` lignes) |
| `find` | Chercher des fichiers : `find /var -name "*.log" -mtime +7` |
| `xargs` | Passer un résultat en arguments : `find ... |
| `tr` | Traduire des caractères : `tr 'a-z' 'A-Z'` |

---

## 9. Codes retour et gestion d'erreurs

```
commande
if [ $? -ne 0 ]; then
 echo "Erreur !" >&2 # Les erreurs vont sur stderr
 exit 1 # Quitter avec un code != 0
fi
# Version courte : ET et OU logiques
mkdir /backup && echo "OK" || echo "Échec"
set -e # Arrêter le script dès la 1ère erreur
set -u # Erreur sur variable non définie
set -o pipefail # Le pipe échoue si une commande du pipe échoue
```

💡 Convention : `0` = succès, `1-255` = erreur. Ton script doit "parler" à ceux qui l'appellent (cron, autres scripts...).

---

## 10. Script interactif : menu type

```
#!/bin/bash
while true; do
 echo "=== MENU ==="
 echo "1) Espace disque"
 echo "2) Utilisateurs connectés"
 echo "q) Quitter"
 read -p "Choix : " choix
 case "$choix" in
 1) df -h ;;
 2) who ;;
 q) break ;;
 *) echo "Choix invalide" ;;
 esac
done
```

---

## 11. Bonus SISR : les réflexes d'administration

```
# Sauvegarde datée (le grand classique)
tar -czf /backup/home_$(date +%F).tar.gz /home
# Vérifier qu'on est root
if [ "$EUID" -ne 0 ]; then echo "Lance-moi en root !"; exit 1; fi
# Logger avec horodatage
echo "$(date '+%F %T') - Sauvegarde terminée" >> /var/log/monscript.log
# Cron (planification) : crontab -e
# ┌─ minute (0-59)
# │ ┌─ heure (0-23)
# │ │ ┌─ jour du mois (1-31)
# │ │ │ ┌─ mois (1-12)
# │ │ │ │ ┌─ jour de la semaine (0-7, 0 et 7 = dimanche)
# 0 2 * * * /opt/scripts/sauvegarde.sh
```

---

## 12. Pièges classiques (à éviter absolument 😅)

1. **Espaces autour du `=`** : `nom = "Alice"` ❌ → `nom="Alice"` ✅
2. **Variables sans guillemets** dans les tests : `[ -f $f ]` casse si le nom contient des espaces.
3. **Parser `ls`** : utilisez `find` ou un glob (`/var/log/*.log`) à la place.
4. **Oublier le shebang** : le script peut tourner avec un autre interpréteur.
5. **`$?` vérifié trop tard** : il est écrasé par la commande suivante, testez-le tout de suite.
6. **CRLF** : un script édité sous Windows a des `\r` invisibles → `dos2unix` ou erreur "bad interpreter".

---

## 13. Check-list du script propre ✅

- [ ] Shebang en première ligne
- [ ] Commentaires d'en-tête (auteur, date, rôle)
- [ ] `set -euo pipefail` pour la robustesse
- [ ] Vérifications des arguments/fichiers avant de bosser
- [ ] Messages d'erreur sur stderr avec `exit` non nul
- [ ] Variables en majuscules pour les constantes
- [ ] Testé avec `bash -x` avant l'entrée en production